# CLAUDE.md â€” Doc-Processor

## INHERITED FROM the Helix Constitution

This module is governed by the Helix Constitution. All rules in the constitutionâ€™s `CLAUDE.md` and the `Constitution.md` it references apply unconditionally. Locate the constitution from any nested depth via its `find_constitution.sh` helper â€” do NOT hardcode a path (this module stays fully decoupled and project-agnostic per Â§11.4.28).

Canonical reference: https://github.com/HelixDevelopment/HelixConstitution

## Module Overview

DocProcessor is a standalone, project-not-aware, fully decoupled Go module (`digital.vasic.docprocessor`, Go 1.25) that loads project documentation, builds structured feature maps, and tracks verification coverage for QA automation. It supports LLM-driven feature extraction via an injected agent and also provides heuristic extraction for offline use. The module imports no consuming-project namespace; project-specific behaviour is injected at runtime through the `Translator`, `LLMAgent`, and `Config` contracts.

## Build & Test

```
go build ./...                          # build all packages + CLI
go test ./...                           # unit + integration + stress + securi
go test ./... -race -count=1            # with race detection
go build -o bin/docprocessor ./cmd/docprocessor
```

`make test`, `make test-race`, and `make test-cover` wrap the above. The CLI runs as `docprocessor [--verbose|-v] <docs-directory>`.

## Package Structure

Seven packages under `pkg/`, plus the CLI under `cmd/docprocessor`:

| Package | Purpose |
|---|---|
| `pkg/loader` | Document loading + parsing (Markdown, YAML, HTML, AsciiDoc, RST) |
| `pkg/feature` | Feature extraction + `FeatureMap` building (`DefaultBuilder`) |
| `pkg/coverage` | Thread-safe (`RWMutex`) coverage tracking |
| `pkg/docgraph` | Directed inter-document link graph with JSON/Mermaid export |
| `pkg/llm` | `LLMAgent` interface + prompt templates (no provider dependency) |
| `pkg/config` | Configuration loading from `.env` files / maps |
| `pkg/i18n` | `Translator` contract + `NoopTranslator` default |

# Key Interfaces

- `loader.Loader` â€” load `loader.Document` values from the filesystem; `SupportedFormats()` reports handled extensions.
- `feature.FeatureMapBuilder` â€” `NewBuilder(projectRoot)` returns a `DefaultBuilder`; `BuildFromDocs(ctx, docs)` produces a `*FeatureMap`.
- `coverage.CoverageTracker` â€” `NewTracker()`; concurrency-safe verification status tracking.
- `llm.LLMAgent` â€” injected LLM for intelligent extraction; no hard dependency on any provider.
- `i18n.Translator` â€” externalised user-facing strings; `NoopTranslator` returns the message ID verbatim as a loud fallback.
- `config.Config` â€” `LoadFromEnv(path)` / `LoadFromMap(env)`.

# Module-Specific Conventions

- Decoupling: never import a consumer namespace under `pkg/**` or `cmd/**`.
- All user-facing CLI output is emitted through the injected `Translator` (message IDs live in `pkg/i18n/bundles/active.en.yaml`), never as hardcoded English literals.
- `.env` is git-ignored and must be `chmod 600`; only `.env.example` is committed.
- `go build ./...` and `go vet ./...` must pass with zero warnings.